

Chapter 6 High-Level and Low-Level Design

This chapter presents the design of the system. It gives an overview, lists the design considerations, and describes the architecture at system and subsystem level with interaction scenarios. It then explains the main architectural strategies, the domain model, and the policies and tactics that govern the implementation.

6.1 System Overview

The system is a research platform made of four subsystems. The data harmonisation pipeline (the `hipfx` package, already implemented) turns raw sources into client datasets. The federated learning simulator trains the shared backbone across the clients. The personalisation and evaluation component trains subtype heads and computes metrics. The demonstration interface serves predictions. The subsystems communicate only through files on disk: the configuration, the exported client folders, model checkpoints and metrics. Each subsystem can therefore be run, tested and replaced on its own.

6.1.1 Data Harmonisation Pipeline

A command-line tool, `hipfx`, runs named stages in a fixed order, each reading the outputs of earlier stages and writing to its own work folder. The command `hipfx all` runs every stage that is currently possible and reports what is blocking the rest.

6.1.2 Federated Learning Simulator

A server process holds the global model and the strategy, and one client process per data source trains locally. Each client is given only the path of its own exported folder.

6.1.3 Demonstration Interface

A web backend loads a chosen model, runs femur detection, cropping and fracture localisation on an uploaded image, and returns the predicted regions and classes to a browser front end.

6.2 Design Considerations

The design must handle sensitive data, heterogeneous sources and limited compute, while producing results that can be reproduced and traced. The considerations below shaped it.

6.2.1 Assumptions and Dependencies

- Software: the Ultralytics YOLOv8 library for the helper models, detection frameworks for the candidate federated models, PyTorch with CUDA, EasyOCR for text detection and a federated learning framework for the simulation. A change in a library's training interface would affect the models built on it.
- Operating system: Linux on the development machines, the team GPUs and the university GPU server.
- Hardware: NVIDIA GPUs (16 GB on the university server, 8–12 GB on team machines); the pipeline also runs on a CPU, more slowly.
- Data: the raw datasets are available locally in the configured paths; the González class mapping and the institutional spreadsheet policy are correct.
- Users: researchers are comfortable with the command line; clinicians use only the web interface.
- Likely changes: new clients (for example KMITL, if its classes are confirmed), new strategies, and a switch from simulation to real deployment, which the file-based client interface is designed to allow.

6.2.2 General Constraints

- Privacy and regulation: institutional images must stay on local or university machines, no derived image may contain burned-in identifiers, and pseudonym maps and galleries are excluded from version control.
- Memory and compute: each simulation runs on one machine, so 1024-pixel inputs and several simulated clients must fit in the memory of a single GPU; clients are trained one after another within a round. Runs with the institutional client must fit the 16 GB university GPU, while runs on public data may use 8–12 GB GPUs with smaller batches.
- Interoperability: every client uses the same folder layout and YOLO-seg label format, so any client can be read by the same code.
- Verification: each stage verifies its outputs (for example, the export re-reads masks and labels), and unit tests cover label parsing, geometry and source adapters.
- Performance: interactive inference must meet the latencies in Chapter 4; training time is bounded by the available GPUs.

6.2.3 Goals and Guidelines

- Keep it simple: Plain files and small functions instead of a database or a service layer, because the data is small and must be easy to audit.
- Provenance over convenience: Every label records where it came from, even when this makes the code longer, because the validity of the results depends on knowing which labels are ground truth.
- Fail loudly: Missing inputs, unknown labels and unreadable files stop the stage or are reported, rather than being silently dropped.
- Configuration, not code: Thresholds, class maps and policies live in the configuration file so that experiments can change them without code changes.
- Privacy by default: Outputs that could contain patient data are written only to local, ignored folders.

6.2.4 Development Methods

The project follows an iterative, research-driven process with short cycles: audit a source, find a problem in the data, design a stage or policy to handle it, verify with a gallery or a test, and repeat. The work is tracked in two-week iterations reviewed with the supervisor. A strict waterfall process was rejected because the problems in the data were only discovered by inspecting it, which changed the method (for example, the move to a per-femur format and the parking of KMITL). The code is developed with unit tests for the parts where correctness can be checked automatically (label parsing, geometry, policies) and visual quality-control galleries for the rest.

6.3 System Architecture

Figure 6.1 shows the layered architecture. The data layer holds the raw sources, the configuration and a private folder. The harmonisation layer turns sources into client datasets. The learning layer contains the federated simulator and the personalisation step, and the evaluation layer compares models. The presentation layer serves the demonstration interface. Each layer depends only on the files produced by the layer below it. This decomposition was chosen because data preparation is by far the largest and most sensitive part of the work, and isolating it lets the learning code treat every client identically. A monolithic training script that read raw sources directly was rejected because it would mix privacy handling with model code and make experiments hard to reproduce.

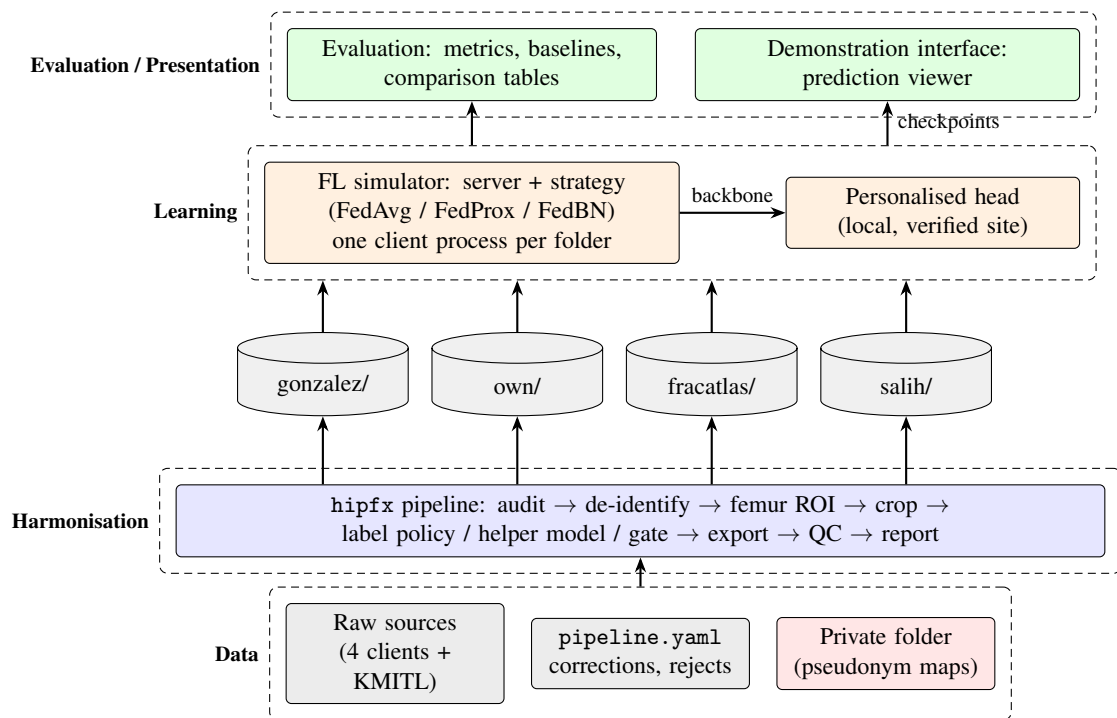


Figure 6.1: Layered system architecture, read from bottom to top. The data layer holds the raw sources, the configuration files and a private folder for pseudonym maps. The harmonisation layer is the hipfx pipeline, whose stages run from audit to report. It writes four client folders (gonzalez, own, fracatlas, salih). The learning layer reads these folders: the federated simulator runs a server with the chosen strategy and one client process per folder, and passes the backbone to the local personalised head. At the top, the evaluation component produces metrics and comparisons, and the demonstration interface loads checkpoints to show predictions on the radiograph.

6.3.1 Subsystem Architecture

Harmonisation pipeline: Figure 6.2 shows the stages of `hipfx` and their dependencies. Source-specific logic is kept in adapter modules (one per source) that turn each dataset into common records with image path, group, labels and flags. All later stages work on these common records. The femur ROI subsystem has a manual step: seed images are labelled locally, imported, and used to train the detector. Until then, sources that need femur boxes wait while the others continue.

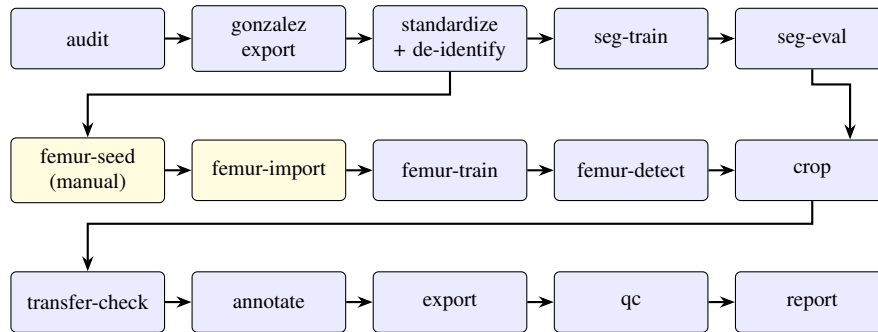


Figure 6.2: Stages of the `hipfx` harmonisation pipeline, in three rows. First row: audit, González export, standardise with de-identification, helper-model training and helper-model evaluation. Second row, which begins with a manual step shown in yellow: femur seed labelling, import of the labels, femur detector training, femur detection and cropping; the trained helper model also feeds the cropped data. Third row: transfer check, annotation, export, quality-control galleries and the final report.

Federated simulator: The server holds the global weights, the strategy object and the round loop. Each client wraps the training loop of the chosen model: it receives weights, loads them (all of them, or all but batch-normalisation for FedBN), trains for E epochs on its folder, and returns its weights and sample count. The strategy decides which parameters are shared and how they are combined.

Demonstration interface: The backend exposes a prediction endpoint that runs femur detection, cropping (with the same code as the pipeline, so inference sees the same format as training) and the personalised model, and returns the predicted regions in image coordinates, classes and confidences. The front end draws the overlays and lists the findings per femur.

6.3.2 Sequence Diagrams

Figures 6.3 and 6.4 show the two central interactions: one federated round and one prediction request.

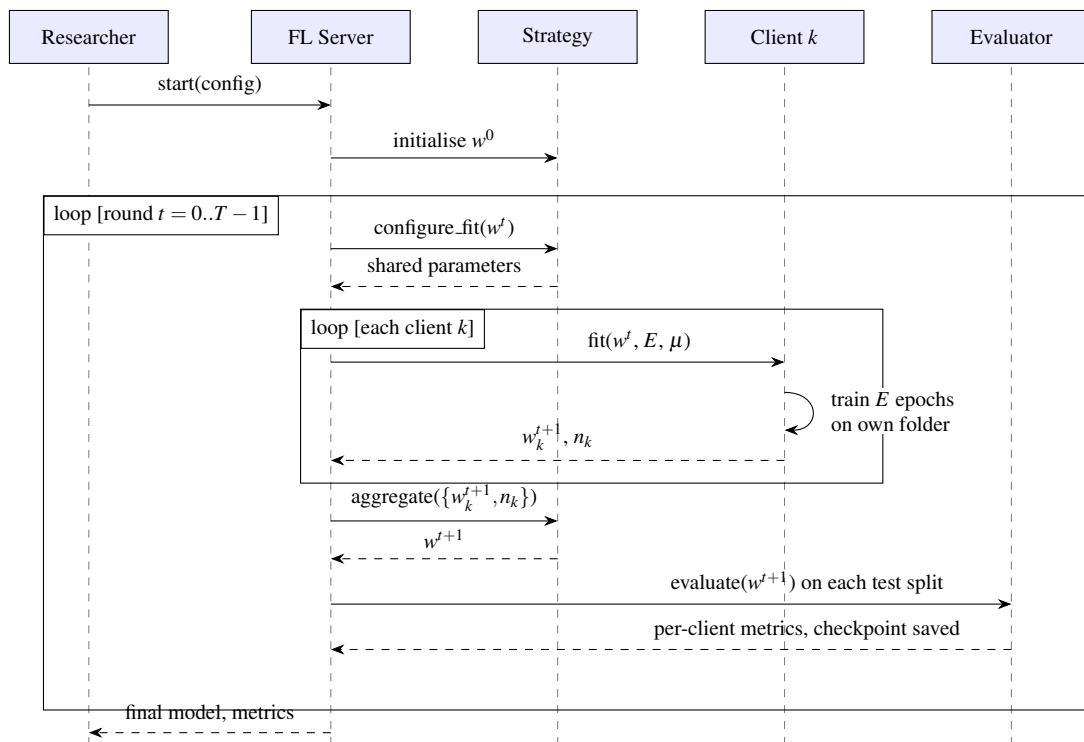


Figure 6.3: Sequence diagram of a federated experiment with five participants: Researcher, FL Server, Strategy, Client k and Evaluator. The researcher starts the experiment with a configuration, and the server asks the strategy to initialise the global weights. In a loop over rounds, the server obtains the parameters to send from the strategy and, in an inner loop over clients, sends the global weights to each client. Each client trains for E epochs on its own folder and returns its weights and sample count. The server passes all updates to the strategy, which returns the aggregated weights. The evaluator then evaluates them on every client's test split and saves a checkpoint. At the end the server returns the final model and metrics to the researcher.

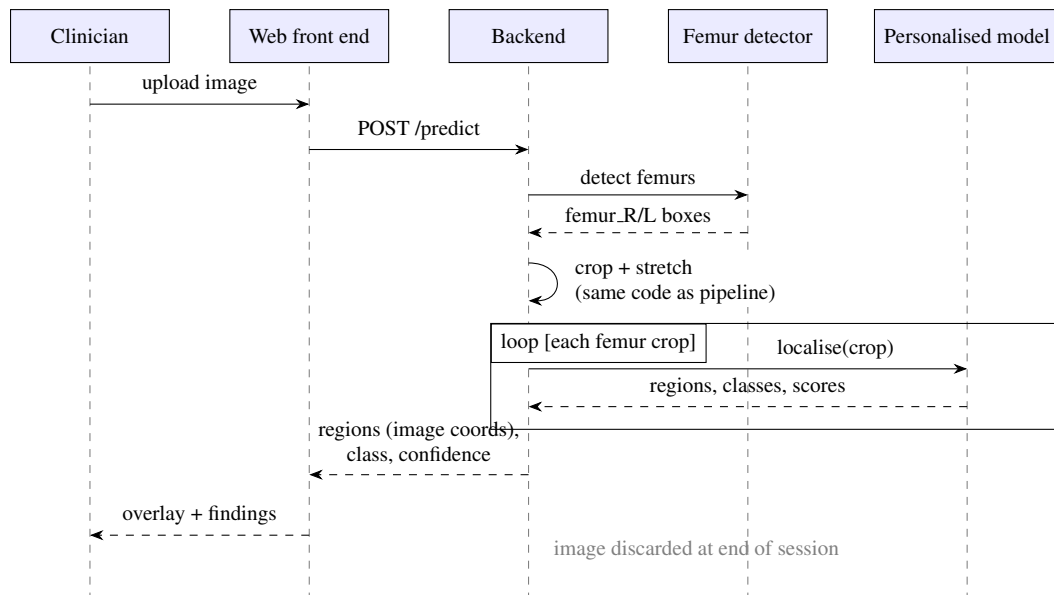


Figure 6.4: Sequence diagram of a prediction request with five participants: Clinician, Web front end, Backend, Femur detector and Personalised model. The clinician uploads an image and the front end sends it to the backend. The backend asks the femur detector for the right and left femur boxes, then crops and stretches each femur with the same code as the pipeline. In a loop over crops, the personalised model returns regions, classes and scores. The backend returns the predicted regions in image coordinates with classes and confidences, and the front end shows the overlay and findings. The image is discarded at the end of the session.

6.4 Architectural Strategies

The strategies below have the largest effect on the structure of the system. For each, the alternatives considered are given.

6.4.1 Resumable, Configuration-Driven Stages

Each pipeline stage writes to its own work folder and a completion marker; completed stages are skipped unless the run is forced. All thresholds, class maps and policies come from one YAML file. This allows the expensive stages (OCR, model training) to run once while later stages are iterated many times, and it records exactly which settings produced a dataset. The alternatives were a single script, rejected because any change would rerun everything, and a workflow engine such as a DAG scheduler, rejected as unnecessary for about fifteen stages on one machine.

6.4.2 Simulated Cross-Silo FL on a Single Machine

Clients run as separate processes on one machine, each given only its own folder, and the server exchanges only weights with them. Independent experiments, such as different strategies or seeds, run in parallel on different machines, but the clients of one experiment are never split across machines, so that differences in hardware do not mix with the effects being studied. The proposed framework is Flower

[36], whose simulation engine runs many clients on one machine, whose strategy interface fits FedAvg and FedProx directly and FedBN with a small custom strategy, and whose client code can later be deployed unchanged. NVIDIA FLARE [35] was considered but is heavier than needed for simulation. A hand-written loop is the fallback if Flower’s integration with the Ultralytics trainer proves awkward; because the client interface (receive weights, train, return weights and count) is the same, the choice does not affect the rest of the design. A real deployment of the same client code across several machines, using only public data, is a possible extension but is not part of the evaluated method.

6.4.3 Privacy by Design

De-identification is applied before any derived image is written. Identifying filenames are replaced by hashes whose mapping is stored only in a private folder, and quality-control galleries are written to a folder excluded from version control. This is a structural decision, not a filter added at the end, because a leak at any intermediate step would already be a breach. Differential privacy and secure aggregation were considered but deferred: they protect model updates rather than the images themselves, which is less important in a simulation that never transmits data outside the machine it runs on.

6.4.4 Two-Stage Shared/Personal Model

The model is split into a backbone shared through FL and a subtype head trained locally. This lets clients with only binary labels still contribute to the representation, and lets the site with subtype labels adapt the classifier to its own class mix. A single federated four-class model was rejected because the binary clients would supply no subtype supervision, or worse, model-derived subtypes treated as ground truth. Fully local models were rejected because the small site alone lacks the data to learn a good representation.

6.5 Domain Model/Class Diagram

Figure 6.5 shows the main classes. `Ctx` carries the configuration and resolves every path. Source adapters turn datasets into common records, and `FemurROI`, `SegModel`, `Annotator` and `Exporter` implement the pipeline stages. The learning side is organised around `FLServer`, `FLClient` and a Strategy hierarchy with `FedAvg`, `FedProx` and `FedBN` subclasses. `PersonalHead` adds local subtype heads and `Evaluator` computes metrics.

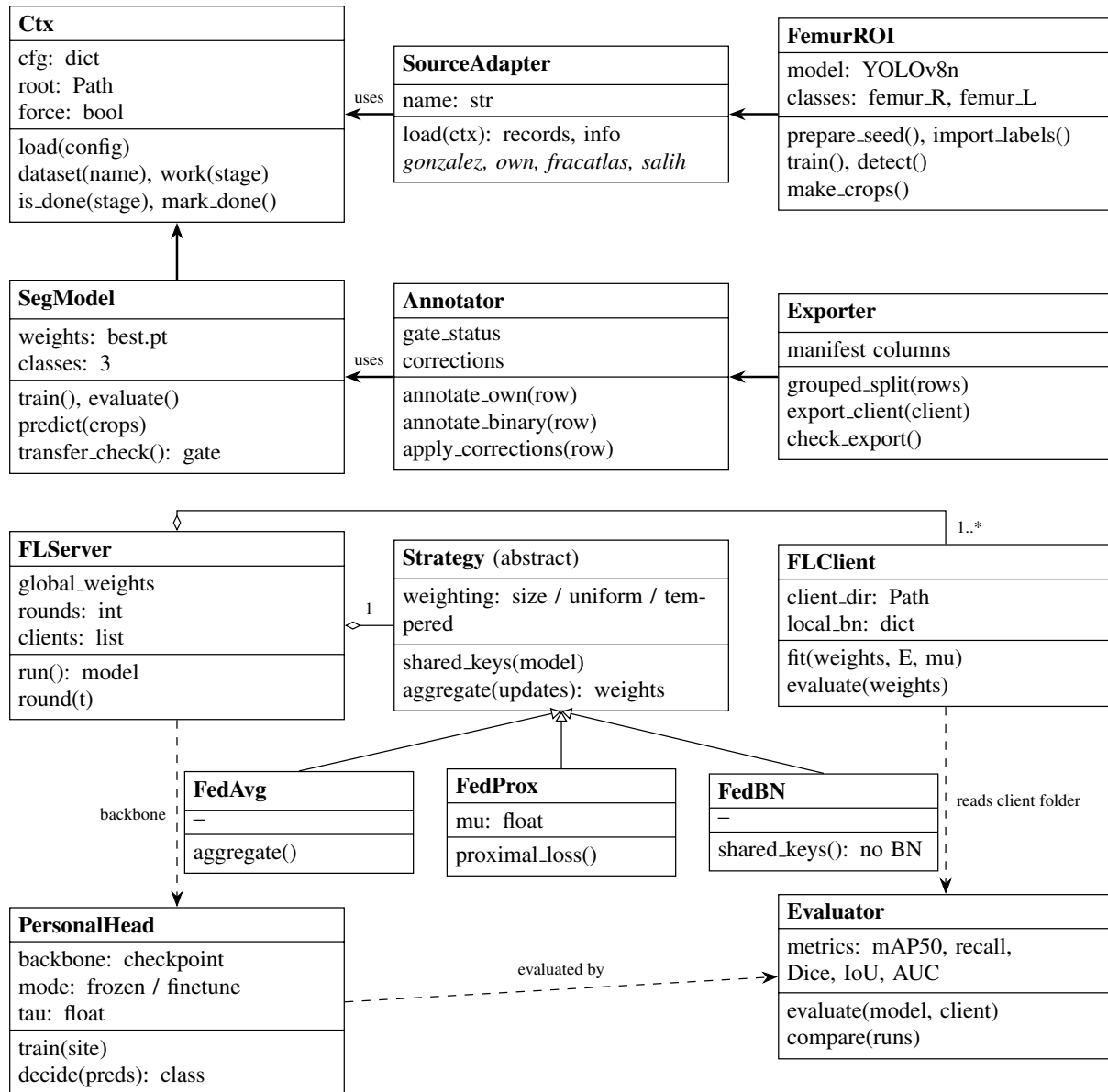


Figure 6.5: Class diagram. The top two rows are the harmonisation pipeline: **Ctx** (configuration and paths) is used by the **SourceAdapter** modules and by **SegModel**; **FemurROI** uses the adapters; **Annotator** uses **SegModel**; and **Exporter** uses **Annotator**. The third row is the federated simulator: **FLServer** aggregates one **Strategy**, which has the subclasses **FedAvg**, **FedProx** (with the parameter **mu**) and **FedBN** (which excludes batch-normalisation parameters from sharing), and the server works with one or more **FLClient** objects, each bound to one client folder. At the bottom, **PersonalHead** takes the backbone from the server and is evaluated by **Evaluator**.

6.6 Policies and Tactics

These policies and tactics affect the details of the implementation rather than its overall structure.

6.6.1 Choice of Tools

Python 3.12 with uv and a lock file gives reproducible environments on every workstation and on the university server. Ultralytics YOLOv8 was chosen for the helper models of the data pipeline because the reference study validated it on the same task and format and because it trains quickly; the architecture of the federated model is left open and will be chosen by comparing candidates such as YOLO-family and Mask R-CNN models. EasyOCR was chosen for text detection because it runs locally, with no cloud service, and handles the fonts of the PACS overlays. Label Studio is used locally for the one-time femur seed labelling; web-hosted labelling tools are not allowed for institutional images. Two build variants of OpenCV that conflict are resolved in the project configuration by keeping only the one required by Ultralytics.

6.6.2 Coding Guidelines and Conventions

The code follows PEP 8, checked with the ruff linter. Functions are small and typed; each module starts with a docstring stating the stage it implements; paths are always resolved through Ctx and never hard-coded; and constants that affect results are placed in the configuration file.

6.6.3 Testing

Unit tests with pytest cover image reading (including truncated files), YOLO-seg label reading and writing, geometry helpers, the institutional spreadsheet policy, source adapters and femur ROI logic. Stages that cannot be checked automatically produce quality-control galleries that are reviewed by hand. For the learning code, a smoke test will run one round with two tiny clients to check that FedBN keeps batch-normalisation parameters local and that aggregation preserves parameter shapes. Full test cases will be specified as the learning code is built.

6.6.4 Requirements Traceability

Every exported crop carries its `class_source`, `geometry_source`, confidence and review flag, so every reported metric can be recomputed on ground-truth-only subsets. Every experiment saves its configuration with its metrics and checkpoints. Functional requirements (FR-01 to FR-23) map to pipeline stages and learning components, and this mapping will be used for the traceability matrix.

6.6.5 Source Organisation and Build

The pre-annotation pipeline lives in `code/pre-annotation/` and reads the raw datasets from `Dataset/pre-annotation/`. It contains `src/hipfx/` (one module per stage, with `sources/` for the adapters), `configs/` (the pipeline configuration, corrections and rejects), `tests/`, and `reports/` (audit and pipeline reports; galleries are ignored by version control). The finished dataset is written to `Dataset/post-annotation/<client>/`, one image, mask, label and outlined preview per femur crop; intermediate outputs stay in `processed/`. Both are ignored by version control. The environment is created with `uv sync`, the full pipeline runs with `uv run hipfx all`, and single stages with `uv run hipfx <stage>`. The federated and interface code will be added as sibling packages that read only `Dataset/post-annotation/<client>/`.

6.6.6 Maintenance

A new data source is added by writing one adapter module and a configuration entry; a new strategy by subclassing `Strategy`. The pipeline report lists pending manual steps and blocked stages, so that a new team member can see the state of the data at a glance.

6.7 Conclusion

This chapter presented the design of the system. A layered architecture separates the sensitive, source-specific harmonisation work from the learning and presentation layers, which communicate only through standard client folders and checkpoints. The main strategies are resumable, configuration-driven stages, a simulated cross-silo federated setup with a pluggable strategy, privacy by design and a two-stage shared/personal model. The class and sequence diagrams show how the components collaborate, and the policies and tactics fix the tools, conventions, testing and traceability. Together with the approach in Chapter 5, this design is the basis for the implementation and experiments that follow.